

LangGraph Memory Guide

How Memory Works in LangGraph

1. **Short-Term Memory = Graph State** Every LangGraph execution has a State. The state acts as the agent's working memory and is continuously updated as nodes execute.
2. **Memory During a Single Run** All nodes share the same state. Nodes can read and write information, but unless checkpointing is enabled, the state disappears after execution ends.
3. **Checkpoint Memory** Checkpointing allows LangGraph to persist state across interactions. Using a checkpointer such as MemorySaver, LangGraph can reload previous conversation state using a thread_id.

MemorySaver Example from langgraph.checkpoint.memory import MemorySaver

```
memory = MemorySaver() graph = builder.compile(checkpointer=memory)
```

Each conversation is identified by a thread_id.

4. **Persistent Memory** MemorySaver stores data only in memory. For production systems, persistent checkpointers such as PostgresSaver are commonly used so memory survives application restarts.
5. **Long-Term Semantic Memory** LangGraph itself does not automatically create vector memories. Typically it is combined with vector databases such as Chroma, FAISS, Pinecone, or Weaviate.
6. **Memory Across Sessions** User preferences and other important information can be stored in databases or vector stores and retrieved in future sessions.

Memory Layers in LangGraph Long-Term Memory (Vector DB / Database)

↓

Checkpoint Memory (Thread Persistence)

↓

Graph State (Current Execution)

Old LangChain vs LangGraph **Old LangChain:** ConversationBufferMemory, ConversationSummaryMemory, etc. **LangGraph:** State + Checkpointer + Database

Interview Answer In LangGraph, memory is primarily managed through graph state and checkpointing. The graph state acts as short-term working memory during execution, while checkpointers such as MemorySaver or PostgresSaver persist state across interactions using a thread_id. For long-term semantic memory, LangGraph is typically integrated with external vector databases or other storage systems. This design separates working memory (state), conversational persistence (checkpoints), and long-term knowledge (external stores).